PhotonicsAI — AI-Driven Science and Autonomous Laboratories

Homework 1: solutions

Name: Erik Mirzoyan Due 11.10.2026

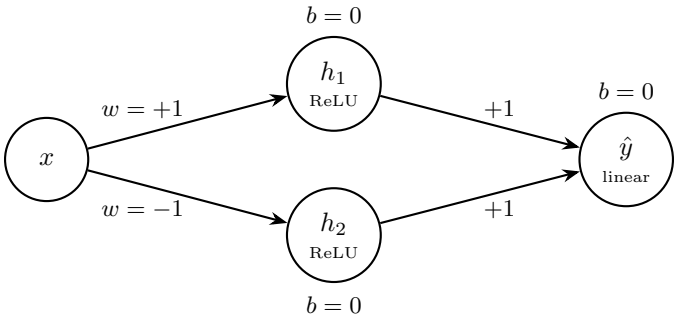
Code repository: `https://github.com/10mirzoyan71/photonicsai-hw1`

Task 1. Two networks a line cannot be

Every hidden unit below is a ReLU unit $h = \mathrm{ReLU}(w^\top x + b)$, every output is linear ($\sigma(z) = z$).

(a) Absolute value (2 hidden ReLU units)

$$h_1 = \mathrm{ReLU}(1 \cdot x + 0), \qquad h_2 = \mathrm{ReLU}(-1 \cdot x + 0), \qquad \hat{y} = 1 \cdot h_1 + 1 \cdot h_2 + 0.$$



For $x \geq 0$: $h_1 = x$, $h_2 = 0$; for $x < 0$: $h_1 = 0$, $h_2 = -x$. In both cases $\hat{y} = |x|$ (two hinges joined at the origin).

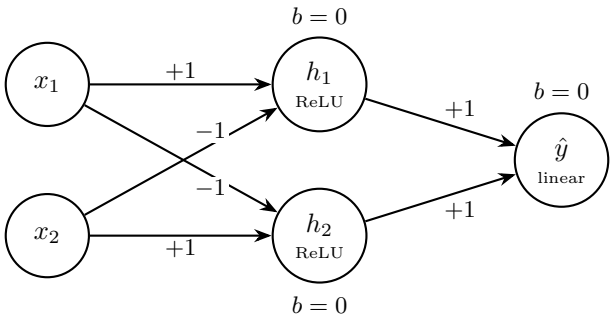
x	$z_1 = x$	h_1	$z_2 = -x$	h_2	$\hat{y} = h_1 + h_2$
-2	-2	0	2	2	2
-0.5	-0.5	0	0.5	0.5	0.5
0	0	0	0	0	0
0.5	0.5	0.5	-0.5	0	0.5
3	3	3	-3	0	3

Worked negative input, $x = -0.5$: unit 1 has pre-activation $z_1 = 1 \cdot (-0.5) + 0 = -0.5$ and activation $h_1 = \max(0, -0.5) = 0$; unit 2 has $z_2 = (-1)(-0.5) + 0 = 0.5$ and $h_2 = \max(0, 0.5) = 0.5$; output $\hat{y} = 0 + 0.5 = 0.5 = |-0.5|$.

Two units are the minimum. With one hidden unit the network is $\hat{y} = c\,\mathrm{ReLU}(wx + b) + d$, a monotone function of x (ReLU is nondecreasing, and c, w only flip directions). $|x|$ decreases on $x < 0$ and increases on $x > 0$, so it is not monotone.

(b) Exclusive or (2 hidden ReLU units, the units of (a) reused on $x_1 - x_2$)

$$h_1 = \mathrm{ReLU}(x_1 - x_2 + 0), \qquad h_2 = \mathrm{ReLU}(-x_1 + x_2 + 0), \qquad \hat{y} = 1 \cdot h_1 + 1 \cdot h_2 + 0 = |x_1 - x_2|.$$



x_1	x_2	$z_1 = x_1 - x_2$	h_1	$z_2 = x_2 - x_1$	h_2	$\hat{y}$	$x_1 \oplus x_2$
0	0	0	0	0	0	0	0
0	1	-1	0	1	1	1	1
1	0	1	1	-1	0	1	1
1	1	0	0	0	0	0	0

Why a single neuron is not enough. The two diagonals of the square have the same midpoint $(\frac{1}{2}, \frac{1}{2})$, so every affine score satisfies $L(0,0) + L(1,1) = L(0,1) + L(1,0)$, so no threshold can put the two 1-corners on one side and the two 0-corners on the other, and a single neuron $\sigma(L(x))$, σ monotone, cannot separate them (XOR is not linearly separable).

Two hidden units are the minimum. With one hidden ReLU unit, $\hat{y} = c\,r(x) + d$ with $r = \mathrm{ReLU}(L(x))$, so r must take one value r_0 on $\{(0,0),(1,1)\}$ and a different value r_1 on $\{(0,1),(1,0)\}$. If $r_0 > 0$, then $L(0,0) = L(1,1) = r_0$ and $L(0,1) + L(1,0) = 2r_0 > 0$, so $r_1 > 0$ and $L(0,1) = L(1,0) = r_1 = r_0$: contradiction. If $r_0 = 0$, then $L(0,0) + L(1,1) \le 0$, while $r_1 > 0$ forces $L(0,1) + L(1,0) = 2r_1 > 0$: contradiction with the midpoint identity.

Task 2. How many parameters are in this network?

Output side $H_{\text{out}} = \lfloor (H + 2p - k)/s \rfloor + 1$; a convolution has $k^2 C_{\text{in}} C_{\text{out}} + C_{\text{out}}$ parameters, a dense layer $N_{\text{in}} N_{\text{out}} + N_{\text{out}}$; ReLU and max-pool have none.

Layer	side computation	output shape	parameter computation	parameters
input		$224 \times 224 \times 3$		0
C1	$\lfloor (224 + 6 - 7)/2 \rfloor + 1 = 112$	$112 \times 112 \times 64$	$7^2 \cdot 3 \cdot 64 + 64$	9 472
P1	$\lfloor (112 - 3)/2 \rfloor + 1 = 55$	$55 \times 55 \times 64$	—	0
C2	$\lfloor (55 + 4 - 5)/1 \rfloor + 1 = 55$	$55 \times 55 \times 192$	$5^2 \cdot 64 \cdot 192 + 192$	307 392
P2	$\lfloor (55 - 3)/2 \rfloor + 1 = 27$	$27 \times 27 \times 192$	—	0
C3	$\lfloor (27 + 2 - 3)/1 \rfloor + 1 = 27$	$27 \times 27 \times 384$	$3^2 \cdot 192 \cdot 384 + 384$	663 936
C4	27	$27 \times 27 \times 256$	$3^2 \cdot 384 \cdot 256 + 256$	884 992
C5	27	$27 \times 27 \times 256$	$3^2 \cdot 256 \cdot 256 + 256$	590 080
P3	$\lfloor (27 - 3)/2 \rfloor + 1 = 13$	$13 \times 13 \times 256$	—	0
F6	flatten $13 \cdot 13 \cdot 256 = 43\,264$	512	$43\,264 \cdot 512 + 512$	22 151 680
F7		256	$512 \cdot 256 + 256$	131 328
F8		10	$256 \cdot 10 + 10$	2 570
Total				**24 741 450**

Convolutions hold $9\,472 + 307\,392 + 663\,936 + 884\,992 + 590\,080 = 2\,455\,872$ parameters; the dense layers hold $22\,151\,680 + 131\,328 + 2\,570 = 22\,285\,578$. The first dense layer alone is 89.5% of the network.

Task 3. Softmax is a probability mass function

(a)

Fix any $z \in \mathbb{R}^K$. The exponential maps $\mathbb{R}$ into $(0, \infty)$, so $e^{z_j} > 0$ for every j, and the normaliser $D = \sum_{j=1}^{K} e^{z_j}$ is a finite sum of K positive real numbers, hence $0 < D < \infty$. Therefore

$$s_i(z) = \frac{e^{z_i}}{D} > 0 \qquad \text{for every class } i = 1, \dots, K.$$

Summing over the classes, the common denominator comes out of the sum:

$$\sum_{i=1}^{K} s_i(z) = \frac{1}{D} \sum_{i=1}^{K} e^{z_i} = \frac{D}{D} = 1.$$

(For $K \geq 2$ this also gives $s_i < 1$.) The vector $s(z)$ is nonnegative (indeed positive) and sums to one over $\{1, \dots, K\}$, which is exactly the definition of a probability mass function on the K classes.

(b)

The normalisation in (a) is the finite sum $\sum_{j=1}^{K}$ over the class index, and the output is one number per class: s is a function on the finite set $\{1, \dots, K\}$, and the probability of a set of classes A is $\sum_{i \in A} s_i$. A density is a function $f(t)$ of a continuous outcome t, normalised by $\int f(t)\, dt = 1$, and gives probabilities as integrals $P(t \in A) = \int_A f(t)\, dt$. Softmax has no continuous outcome in it: its argument z is the vector of scores, not the outcome, and the outcome is the class label. If the class probabilities came from a density on a continuous line, every single outcome would have probability $\int_{\{i\}} f(t)\, dt = 0$, because a point has length zero; but by (a) softmax gives each class the strictly positive probability $s_i > 0$. So s is not a density: its normalisation is a sum over the classes, not an integral over an outcome.

Task 4. A dot product does not know the seat

Notation: $q_i^\top, k_i^\top, v_i^\top$ are the i-th rows of Q, K, V; $(P)_{i,\pi(i)} = 1$, so $(PM)_{i,:} = M_{\pi(i),:}$ and $(MP^\top)_{:,j} = M_{:,\pi(j)}$ for any matrix M with n rows/columns.

(a) The score is a dot product

$(QK^\top)_{ij} = \sum_l Q_{il} K_{jl} = q_i^\top k_j$. After reordering, $Q' = (PX)W_Q = P(XW_Q) = PQ$ and likewise $K' = PK$, so $q'_i = q_{\pi(i)}$ and $k'_j = k_{\pi(j)}$: the query in seat i is the old query of the token that moved there. Hence

$$(q'_i)^\top k'_j = q_{\pi(i)}^\top k_{\pi(j)}, \qquad \text{i.e.} \qquad Q' K'^\top = P\,(QK^\top) P^\top.$$

Follow one token that moves from seat 1 to seat 3 ($\pi(3) = 1$): the dot products of its query with the other tokens' keys are the same numbers as before, they are now found in row 3 instead of row 1 (and its key's dot products in column 3 instead of column 1).

Two tokens, $u = (1, 0)$, $v = (0, 1)$, $W_Q = W_K = I$, so the scores are dot products of the rows.

$X = \binom{u}{v}$	seat 1 (u)	seat 2 (v)
seat 1 (u)	$u \cdot u = 1$	$u \cdot v = 0$
seat 2 (v)	$v \cdot u = 0$	$v \cdot v = 1$

$PX = \binom{v}{u}$	seat 1 (v)	seat 2 (u)
seat 1 (v)	$v \cdot v = 1$	$v \cdot u = 0$
seat 2 (u)	$u \cdot v = 0$	$u \cdot u = 1$

The four numbers are the same. What moved: $u \cdot u$ went from entry $(1, 1)$ to $(2, 2)$, $v \cdot v$ from $(2, 2)$ to $(1, 1)$, $u \cdot v$ from $(1, 2)$ to $(2, 1)$ and $v \cdot u$ from $(2, 1)$ to $(1, 2)$. (With these orthonormal tokens the two matrices even look identical; the labels show that every entry changed seat.)

(b) The whole layer follows the tokens

Let $S = QK^\top/\sqrt{d_k}$ and $A = \mathrm{softmax}_{\mathrm{row}}(S)$. By (a), the reordered scores are $S' = PSP^\top$, i.e. $S'_{ij} = S_{\pi(i)\pi(j)}$. Row i of S' is row $\pi(i)$ of S with its entries reordered by π. Softmax acts on each row separately and the softmax of a reordered vector is the reordered softmax, so

$$A'_{ij} = \mathrm{softmax}(S'_{i,:})_j = \mathrm{softmax}(S_{\pi(i),:})_{\pi(j)} = A_{\pi(i)\pi(j)}, \qquad A' = PAP^\top.$$

With $V' = PXW_V = PV$ and $P^\top P = I$:

$$\mathrm{Att}(PX) = A'V' = PAP^\top PV = PAV = P\,\mathrm{Att}(X).$$

So the output row that belongs to a given token is the same vector whatever seat the token sits in; it simply moves with the token to its new row.

Two tokens ($W_V = I$, factor $1/\sqrt{d_k}$ dropped). The score matrix is I in both orders, and $\mathrm{softmax}(1,0) = (\frac{e}{1+e}, \frac{1}{1+e}) = (0.7311,\ 0.2689)$:

$$\mathrm{Att}(X) = \begin{pmatrix} 0.7311 & 0.2689 \\ 0.2689 & 0.7311 \end{pmatrix}\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} 0.7311 & 0.2689 \\ 0.2689 & 0.7311 \end{pmatrix} \begin{matrix} \leftarrow u \\ \leftarrow v \end{matrix}$$

$$\mathrm{Att}(PX) = \begin{pmatrix} 0.7311 & 0.2689 \\ 0.2689 & 0.7311 \end{pmatrix}\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} 0.2689 & 0.7311 \\ 0.7311 & 0.2689 \end{pmatrix} \begin{matrix} \leftarrow v \\ \leftarrow u \end{matrix} = P\,\mathrm{Att}(X).$$

Token u receives $(0.7311, 0.2689)$ in both orders, token v receives $(0.2689, 0.7311)$; only the rows changed places.

(c) Position has to enter before the dot product

Let P swap seats 1 and 2 and let E have rows e_1, e_2 (and $e_3, \ldots, e_n$, which P leaves in place). Then

$$PX + E = \begin{pmatrix} x_2 + e_1 \\ x_1 + e_2 \\ x_3 + e_3 \\ \vdots \end{pmatrix}, \qquad P(X + E) = PX + PE = \begin{pmatrix} x_2 + e_2 \\ x_1 + e_1 \\ x_3 + e_3 \\ \vdots \end{pmatrix},$$

$$(PX + E) - P(X + E) = E - PE = \begin{pmatrix} e_1 - e_2 \\ e_2 - e_1 \\ 0 \\ \vdots \end{pmatrix} \neq 0$$

whenever $e_1 \neq e_2$. So the two inputs differ, and they differ in the very vectors that enter the dot products: the token that moved to seat 1 now enters as $x_2 + e_1$ instead of carrying its old $x_2 + e_2$, so e.g. its self-score becomes $(x_2 + e_1)^\top(x_2 + e_1)$ instead of $(x_2 + e_2)^\top(x_2 + e_2)$ — the numbers change, not only their seats. A stack F of attention layers and of layers that apply one function to every row still satisfies $F(PZ) = PF(Z)$ for any input Z (part (b), and row-wise maps commute with row permutations). Applied to $Z = X + E$ this says that "the tokens moved with their encodings" gives $F(P(X + E)) = PF(X + E)$: nothing new. The model, however, receives $PX + E \neq P(X + E)$, so in general $F(PX + E) \neq PF(X + E)$: order is visible because the encodings changed the vectors before any dot product was taken.

Example (u, v as above, $e_1 = (1, 0)$, $e_2 = (0, 0)$, $W_Q = W_K = W_V = I$, no scaling): $X + E$ has rows $(2, 0), (0, 1)$ and scores $\left(\begin{smallmatrix} 4 & 0 \\ 0 & 1 \end{smallmatrix}\right)$; $PX + E$ has rows $(1, 1), (1, 0)$ and scores $\left(\begin{smallmatrix} 2 & 1 \\ 1 & 1 \end{smallmatrix}\right)$, which is not $P\left(\begin{smallmatrix} 4 & 0 \\ 0 & 1 \end{smallmatrix}\right)P^\top = \left(\begin{smallmatrix} 1 & 0 \\ 0 & 4 \end{smallmatrix}\right)$. The outputs are $\mathrm{Att}(PX + E) = \left(\begin{smallmatrix} 1.000 & 0.731 \\ 1.000 & 0.500 \end{smallmatrix}\right)$ versus $P\,\mathrm{Att}(X + E) = \left(\begin{smallmatrix} 0.538 & 0.731 \\ 1.964 & 0.018 \end{smallmatrix}\right)$.

Task 5. PINN for the Silverbox oscillator

Code: `https://github.com/10mirzoyan71/photonicsai-hw1`, folder `task5_pinn_silverbox/` (`silverbox_pinn.py`: data, network, RK4, training, metric; `sweep.py`: all runs below; `make_figures.py`: plots and this section's numbers). PyTorch, float64.

Data

`SNLS80mV.mat`, $u =$ `V1`, $y =$ `V2`, slice $[42650 : 46746]$ (4096 samples), each signal centred by its mean on the excerpt ($\bar{u} = 6.66\,\mathrm{mV}$, $\bar{y} = 1.32\,\mathrm{mV}$). Train: samples 0–3071 (5.03 s); test: 3072–4095 (1.68 s). The output spectrum peaks at 70.0 Hz, the input band ends at 200 Hz, the RMS of the test output is 60.49 mV (the zero predictor). One property of this split decides the design: **the test segment reaches $|y| = 0.205\,\mathrm{V}$ while the training record never exceeds $0.174\,\mathrm{V}$**, so on the largest test peak the force has to be extrapolated exactly where the cubic term is strongest.

Model

Equation (1), $\ddot{y} + a\dot{y} + by + cy^3 = g\,u(t)$, in state form with $v = \dot{y}$:

$$\dot{y} = v, \qquad \dot{v} = \alpha_\theta(y, v, u),$$

where α_θ is a fully connected network. *Scaling:* the inputs are $(y/\sigma_y,\ v/\sigma_v,\ u/\sigma_u)$ and the raw output is multiplied by σ_a, the standard deviations on the training record ($\sigma_y = 53.0\,\mathrm{mV}$, $\sigma_v = 22.4\,\mathrm{V/s}$, $\sigma_u = 22.0\,\mathrm{mV}$, $\sigma_a = 1.04 \cdot 10^4\,\mathrm{V/s^2}$), so every input and the target are $O(1)$. Architectures tried: A1 2×32 tanh, A2 2×64 tanh, A3 3×64 tanh, A4 2×64 SiLU (all with a linear output).

Integrator: classical RK4 at $\Delta t = 1/f_s$, driven by the measured input: stage 1 uses u_k, stages 2–3 use $u(t_k + \Delta t/2)$, stage 4 uses u_{k+1}. The half-sample values come from a 16-tap Hann-windowed sinc interpolator of the measured u (the input is a multisine up to 200 Hz, where the plain average $(u_k + u_{k+1})/2$ would keep only $\cos(\pi \cdot 200/610) \approx 0.51$ of the amplitude); only u is used, which is known on the whole excerpt.

Training (only the 3072 training samples enter any loss)

1. **Stage A, identify the equation.** The four coefficients of (1) are first fitted to the training record: least squares on finite-difference derivatives as the start, then the same rollout loss as in stage C (Adam, lr $3 \cdot 10^{-3}$, 600 steps, rollout 256). This is run `R_duffing` in the table; it gives $a = 40.95\,\mathrm{s^{-1}}$, $b = 1.849 \cdot 10^5\,\mathrm{s^{-2}}$ ($f_0 = \sqrt{b}/2\pi = 68.4\,\mathrm{Hz}$), $c = 7.474 \cdot 10^5\,\mathrm{V^{-2}s^{-2}}$, $g = 1.861 \cdot 10^5\,\mathrm{s^{-2}}$. At $|y| = 0.2\,\mathrm{V}$ the cubic force is $cy^2/b = 16\%$ of the linear force.
2. **Stage B, residual pre-fit (collocation on the data).** 500 Adam steps on $\|\ddot{y}_{\mathrm{FD}} - \alpha_\theta(y, \dot{y}_{\mathrm{FD}}, u)\|^2/\sigma_a^2$ with 4th-order central differences. It gives the network the right sign and size of the restoring force before any rollout.
3. **Stage C, rollout training + PINN residual.** Each step draws 32 windows at random starts in the training record, starts each at the measured y and the 4th-order finite-difference velocity, integrates with RK4 and penalises

$$\mathcal{L} = \frac{1}{\sigma_y^2}\,\overline{(\hat{y} - y)^2} \;+\; \lambda_{\mathrm{phys}}\,\frac{1}{\sigma_a^2}\,\overline{\big(\alpha_\theta(\tilde{y}, \tilde{v}, \tilde{u}) + a\tilde{v} + b\tilde{y} + c\tilde{y}^3 - g\tilde{u}\big)^2},$$

   the second term on 512 collocation points per step drawn uniformly from a box $1.5\times$ the training maxima of $|y|, |v|, |u|$ (it reaches $|y| = 0.26\,\mathrm{V}$, past the test peak). a, b, c, g are the stage-A values, held fixed: they are not weights of the network, and only the network is integrated at test time. The rollout length grows geometrically from 16 to the final length over the first 40% of the steps; cosine learning-rate decay to 5%, gradient-norm clip 1. Every 100 steps the free run of the training record is computed, and the weights with the lowest *training* free-run RMSE are kept for that run.

Metric

After the weights are fixed: one free run of all 4096 samples from y_0 and the one-sided finite difference $v_0 = (-3y_0 + 4y_1 - y_2)/2\Delta t$, RK4 through the training samples and on through the test samples with the same state, reading u only. RMSE on samples 3072–4095 (test) and on 0–3071 (train).

Results

Kept version: **A2_h5** (64×64 tanh, 4481 weights)
free-run RMSE on the test record (1024 samples): 0.422 mV
free-run RMSE on the training record (3072 samples): 0.407 mV
zero-prediction baseline on the test record: 60.49 mV

Settings of the kept version: input scaling by the training standard deviations, 64×64 tanh hidden layers, linear output scaled by σ_a; RK4 at Δt with sinc-interpolated midpoint input; stage B 500 steps; stage C Adam, learning rate 0.003 with cosine decay, 1000 steps, batch 32 windows, rollout growing from 16 to 128 samples, PINN weight $\lambda_{\text{phys}} = 1$ with the identified coefficients, gradient clip 1; checkpoint at step 1000 (lowest training free-run RMSE).

All runs (sorted by name; highlighted: kept). "train"/"test" are the free-run RMSEs of the metric, in mV. Ak_hj = architecture k with training setting j; L_* = ablation of the physics loss on A2_h2; R_* = references that are not networks (4 coefficients of eq. (1), trained by the same rollout loss; R_linear has $c = 0$).

run	force model	opt.	lr	steps	rollout	λ_{phys}	params	train [mV]	test [mV]
A1_h1	32×32 tanh	adam	0.001	500	128	1	1217	0.686	0.860
A1_h2	32×32 tanh	adam	0.003	500	128	1	1217	0.643	0.799
A1_h3	32×32 tanh	adam	0.003	500	256	1	1217	0.618	0.797
A1_h4	32×32 tanh	rmsprop	0.001	500	128	1	1217	0.897	1.185
A1_h5	32×32 tanh	adam	0.003	1000	128	1	1217	0.464	0.481
A2_h1	64×64 tanh	adam	0.001	500	128	1	4481	0.517	0.685
A2_h2	64×64 tanh	adam	0.003	500	128	1	4481	0.478	0.632
A2_h3	64×64 tanh	adam	0.003	500	256	1	4481	0.487	0.635
A2_h4	64×64 tanh	rmsprop	0.001	500	128	1	4481	0.757	0.900
A2_h5	64×64 tanh	adam	0.003	1000	128	1	4481	0.407	0.422
A3_h1	$64 \times 64 \times 64$ tanh	adam	0.001	500	128	1	8641	0.445	0.617
A3_h2	$64 \times 64 \times 64$ tanh	adam	0.003	500	128	1	8641	0.449	0.656
A3_h3	$64 \times 64 \times 64$ tanh	adam	0.003	500	256	1	8641	0.439	0.594
A3_h4	$64 \times 64 \times 64$ tanh	rmsprop	0.001	500	128	1	8641	0.662	0.934
A3_h5	$64 \times 64 \times 64$ tanh	adam	0.003	1000	128	1	8641	0.418	0.447
A4_h1	64×64 silu	adam	0.001	500	128	1	4481	1.197	2.181
A4_h2	64×64 silu	adam	0.003	500	128	1	4481	0.864	1.721
A4_h3	64×64 silu	adam	0.003	500	256	1	4481	0.739	1.353
A4_h4	64×64 silu	rmsprop	0.001	500	128	1	4481	2.366	4.077
A4_h5	64×64 silu	adam	0.003	1000	128	1	4481	0.499	0.868
L_joint	64×64 tanh	adam	0.003	500	128	1	4481	1.191	4.185
L_pw0	64×64 tanh	adam	0.003	500	128	0	4481	0.725	7.627
L_pw01	64×64 tanh	adam	0.003	500	128	0.1	4481	0.415	0.773
L_pw10	64×64 tanh	adam	0.003	500	128	10	4481	0.695	0.758
R_duffing	eq. (1), 4 coeff.	–	–	600	256	–	4	0.405	0.393
R_linear	eq. (1), 4 coeff. (c=0)	–	–	600	256	–	4	4.961	13.546

What the sweep shows.

- **The physics loss is what makes the network extrapolate.** On A2_h2, without it ($\lambda_{\text{phys}} = 0$, run L_pw0) the network fits the training record (0.725 mV) but reaches 7.627 mV on the test segment; almost all of that error sits on the 0.205 V peak, where the network trained on $|y| \leq 0.174$ V under-stiffens (force-curve figure). With the residual to the identified equation the same network gets 0.632 mV ($\lambda = 1$); $\lambda = 0.1$ gives 0.773 mV and $\lambda = 10$ gives 0.758 mV. Learning a, b, c, g jointly with

the network instead of identifying them first (L_joint) gives 4.185 mV: the coefficients drift towards whatever the network does outside the data (in that run the cubic coefficient ended at $c = 4.14 \cdot 10^5$ against the identified $7.47 \cdot 10^5$), so the residual stops carrying information there.

- **Training length matters most.** In every architecture the best setting is h5 (1000 steps); a rollout of 256 (h3) against 128 (h2) helps for some networks and not for others, and RMSprop (h4) is the worst setting for A1, A2, A3, A4. The training free-run error of the kept run was still going down at the end, so more steps would likely lower it further.
- **Architecture.** Best test RMSE per architecture: 2×32 tanh: 0.481 mV (A1_h5); 2×64 tanh: 0.422 mV (A2_h5); 3×64 tanh: 0.447 mV (A3_h5); 2×64 SiLU: 0.868 mV (A4_h5). The tanh networks with 64 units are better than the 32-unit one at equal settings; SiLU is clearly worse here.
- **Comparison with the bare equation.** Eq. (1) with its four identified coefficients, integrated the same way, scores 0.405 mV train / 0.393 mV test; the linear equation ($c = 0$) scores 4.961 / 13.546 mV. So the cubic term is essential, and on this excerpt the circuit is described by (1) so well that the kept network (0.422 mV) matches the identified equation to within 0.03 mV without beating it. The network's force is free to depart from (1) where the training data ask for it, and the physics term keeps it on (1) where there are no data; the physics read-out below shows the force it settled on.
- **Train/test gap.** 0.407 mV vs 0.422 mV for the kept version: no sign of a drift on the test segment. The version is chosen by test score, as the task asks, so the reported test number is slightly optimistic; inside every run the checkpoint was chosen by the training free-run error only.

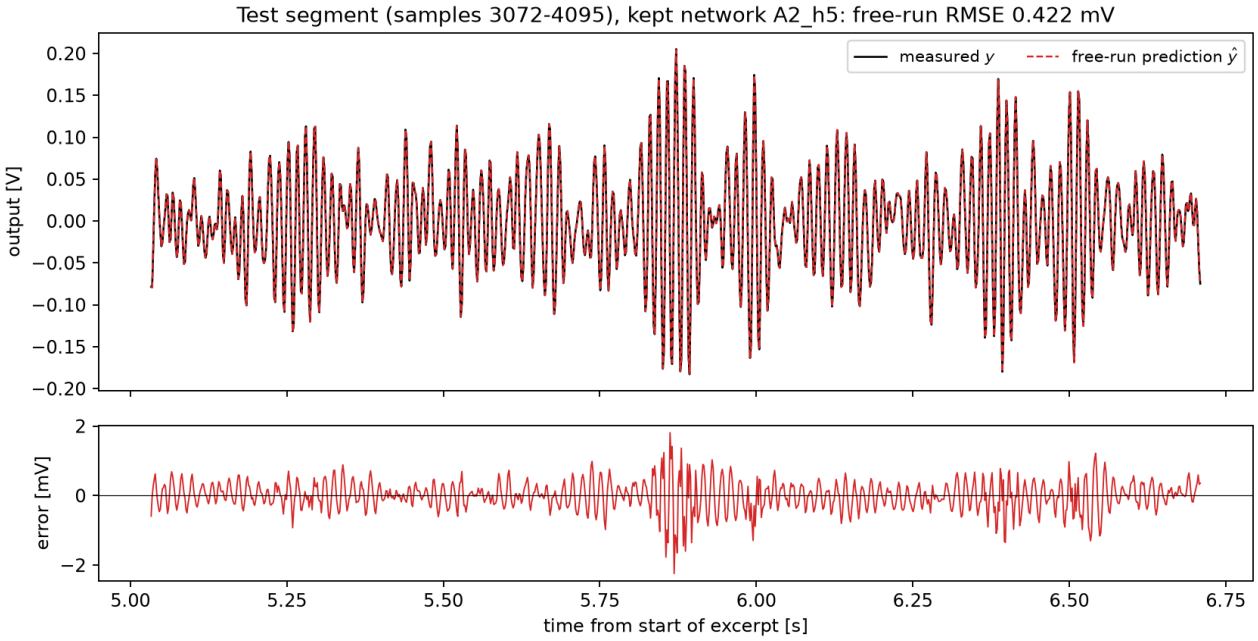


Figure 1: Measured output and free-run prediction of the kept network on the test segment, and the error.

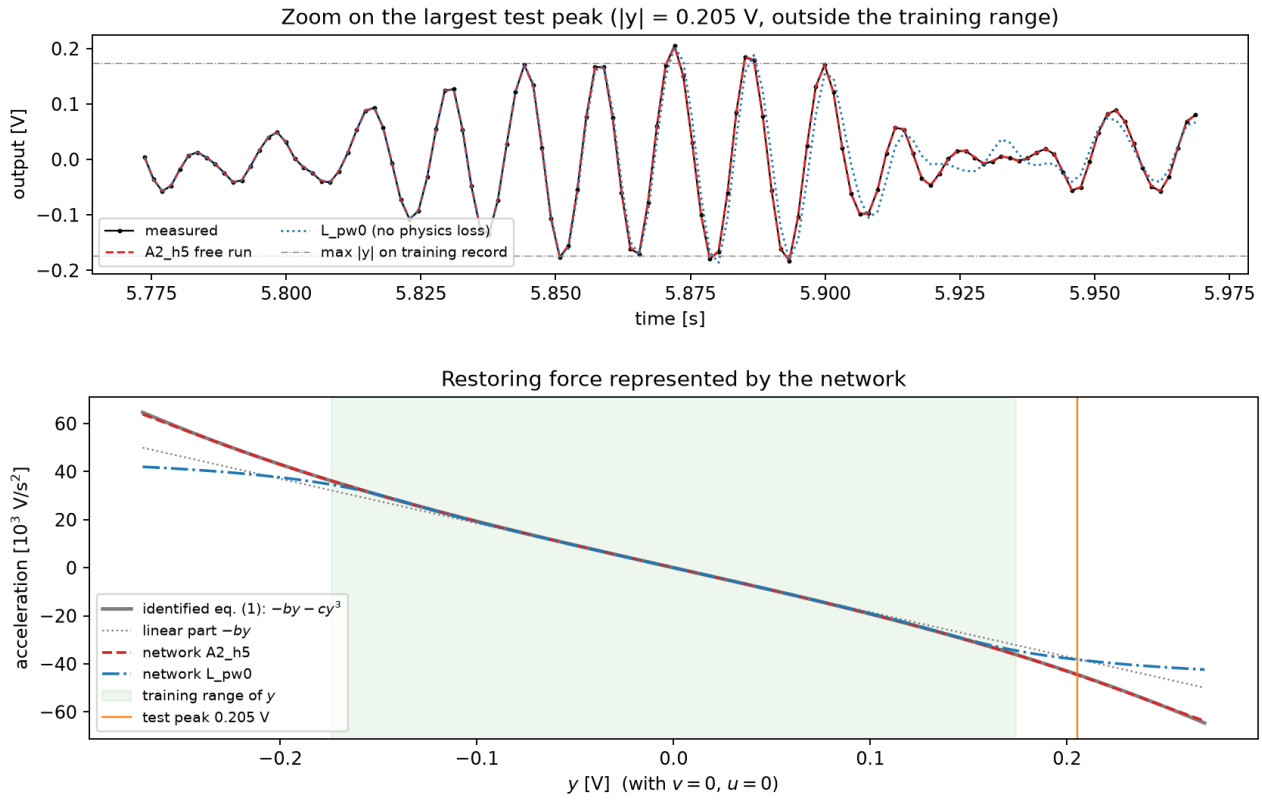
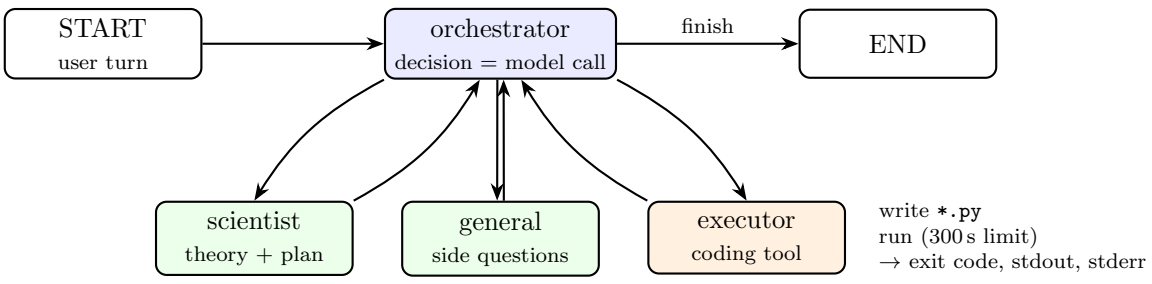


Figure 2: Top: the largest test peak, outside the training amplitude range, with and without the physics loss. Bottom: the restoring force $\alpha_\theta(y, 0, 0)$; the network trained without the physics loss under-stiffens past the training range.

Physics read-out of the kept network. Projecting α_θ on the training states onto $\{-v, -y, -y^3, u\}$ by least squares gives $a = 40.99$, $b = 1.850 \cdot 10^5$ ($f_0 = 68.46\,\mathrm{Hz}$), $c = 7.326 \cdot 10^5$, $g = 1.867 \cdot 10^5$ with $R^2 = 0.99999$: the network represents a Duffing-like force, in line with the stage-A identification.

Task 6. Agentic system

Code: `https://github.com/10mirzoyan71/photonicsai-hw1`, folder `task6_agentic_system/`. Files: `agents.py` (state, prompts, nodes, graph), `coding_tool.py`, `run_task5.py` (gives the system Task 5), `render_transcript.py`, `tests/offline_smoke_test.py`. LangGraph, LangChain; the model behind the roles is chosen with `--model`. The run below used Claude Sonnet 5.5 through the Claude Code CLI (`--model claude-cli:claude-sonnet-5-5`, `claude_cli.py`), with the CLI's own tools switched off: decisions are its structured output, and the executor's coding-tool calls are requested as structured output and run by the executor node.



State. The state of the `StateGraph` is the dialogue: a list of turns, each {`role, content, meta`} (role name, the text it said, and machine-readable fields such as the decision or the tool record), merged with `operator.add`, so every node reads all earlier turns, including what the other agents wrote, and appends exactly one turn. Routing, the decision count and the finish check are all derived from the dialogue itself.

Edges. `START`→orchestrator; a conditional edge from the orchestrator to scientist, general, executor or `END`, chosen by the orchestrator's decision; an edge back from each specialist to the orchestrator.

Orchestrator. A model call with structured output `Decision(next_role, instruction, reason)`, where `next_role` ∈ {scientist, general, executor, finish}. It sees the whole dialogue plus a status line (decision k of 8, whether a verified RMSE exists). It has no tool and its prompt forbids theory and code. Rules enforced in code: at most 8 decisions; `finish` is accepted only if some executor turn holds a `TEST_RMSE` parsed from the standard output of a script that ran with exit code 0 — otherwise the model is asked once more with the reason, and a second `finish` is routed to the executor.

Scientist / general agent. Model calls without tools. The scientist writes the equation, the network, the state equation, the training, the free-run test and a numbered implementation list, and may not quote a number that no executor turn printed. The general agent answers high-level side questions (what a record is, what a term or an error means) and writes no plan.

Executor and the coding tool. The only role bound to the tool `coding_tool(action, filename, code)`: `write` creates a *new* `.py` file in the run's working directory (existing names are refused, so every implementation and every repair is a new file); `run` executes it with a 300 s limit and returns exit code, stdout and stderr. One write and one run per executor turn; the node appends a tool record (files, exit codes, the parsed `TEST_RMSE`/`TRAIN_RMSE` marked VERIFIED, or the stderr tail with the traceback). A later executor turn sees the previous script and can repair it; the plan stays the scientist's.

Validation run on Task 5

Model behind every role: `claude-cli:claude-sonnet-5-5`. Turns: 6; orchestrator decisions: 3.

Free-run error printed by the script that ran (`silverbox_pinn.py`): test RMSE = 0.00100889 V, train RMSE = 0.00144579 V.

#	role	what it did
1	user	Please solve the following task with your team and report the verified free-run errors. Task 5. PINN (Physics Informed Neural Network) for the Silv...
2	orchestrator	-> scientist: A plan must be written by the scientist before any code can be executed.
3	scientist	# Plan for Task 5: Silverbox PINN No numbers have been measured yet. Every size below is an expectation from the problem statement, not a result. #...
4	orchestrator	-> executor: The plan is complete, so it must now be coded and run to obtain verified RMSE values.
5	executor	write silverbox_pinn.py: ok run silverbox_pinn.py: exit code 0, 210.0 s VERIFIED printed by the script: TEST_RMSE=0.0010088887947707441, TRAIN_RMSE=0.0014457...
6	orchestrator	-> finish: A VERIFIED executor result with both TRAIN_RMSE and TEST_RMSE exists, so the run can be finished.

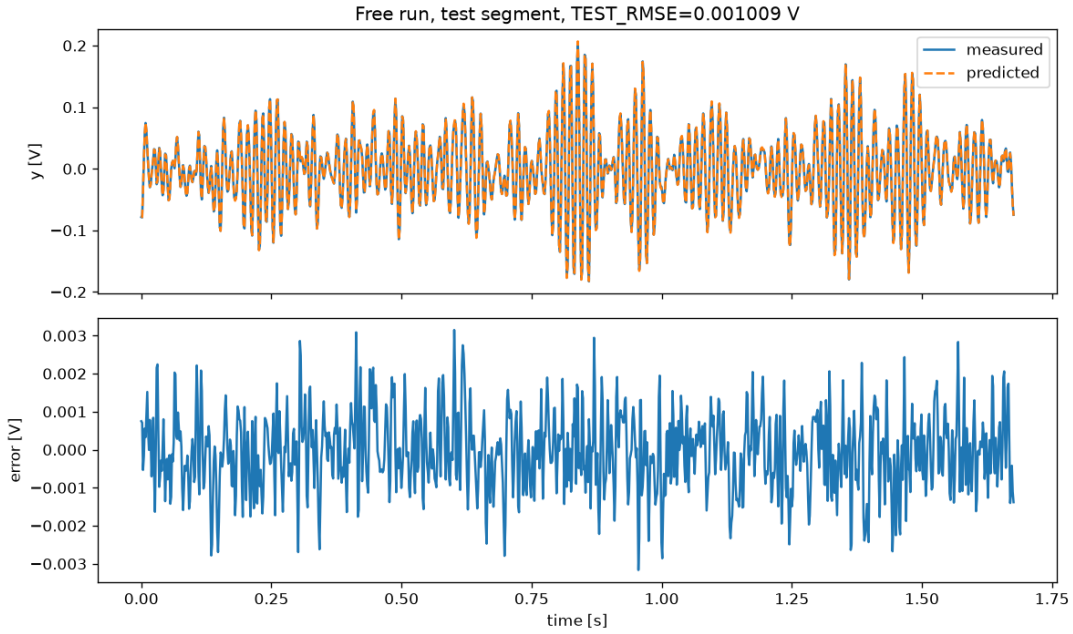


Figure 3: The plot saved by the agents' script: measured output and free-run prediction on the test segment.

Transcript (abridged). Turn 1 — user

```
Please solve the following task with your team and report the verified free-run errors....
[the full Task 5 statement follows: task6_agentic_system/task5_statement.md]
```

Turn 2 — orchestrator

```
Decision 1/8 -> scientist
Instruction: Write the complete, code-ready plan for Task 5 (Silverbox PINN). Include: data loading from SNLS80mV.
     mat (V1=u, V2=y), slice [42650:46746], mean-centering, split 3072 train / 1024 test, dt=1/610.35. Network
     alpha_theta(y,v,u) with chosen depth/widths/activation and input/output scaling, optionally physics-informed
     initialization from eq. (1) (y''=-a v - b y - c y^3 + g u) with learnable a,b,c,g as a residual/prior (b ~
     (2*pi*70)^2, a few tens). RK4 integration with u held/interpolated between samples (specify, e.g. midpoint u
     average), training scheme (multiple shooting over short windows, curriculum on window length, Adam then maybe
      LBFGS, learning rates, epochs), keeping total runtime under 5 minutes on CPU. Then the final free run:
     initial state from the first sample with finite-difference velocity, integrate through train into test
     without reading test y, print TRAIN_RMSE and TEST_RMSE, and save a plot of measured vs predicted on the test
     segment. Give a numbered list of steps specific enough to code.
Reason: A plan must be written by the scientist before any code can be executed.
```

Turn 3 — scientist

```
# Plan for Task 5: Silverbox PINN

No numbers have been measured yet. Every size below is an expectation from the problem statement, not a result.

## 1. Differential equation and coefficients

y'' + a y' + b y + c y^3 = g u(t), with y and u the mean-centered signals.

| Coef | Meaning | Expected size / initial value |
|---|---|---|
| a | damping, 1/s | tens, about 20-40 |
| b | linear stiffness, 1/s^2 | (2pi*70)^2 ~ 1.9e5 |
| c | cubic stiffness, 1/(V^2*s^2) | cy^2 ~ 0.1-0.3*b at |y| = 0.2, so c ~ 5e5-1.5e6 |
| g | input gain, V/(V*s^2) | order 1e4 (from y ~ g*u/(a*omega) at resonance); fix it by regression |

- a, b, c and g are four learnable scalars of the physics prior. They are not network weights.
- Initialize them by least squares on the training record (step 4), then refine them by shooting.
- The RK4 step at 70 Hz has omega*dt ~ 0.72, so the integrator carries phase and damping error of its own.
     Training and testing use the same integrator, so the fitted b and a will absorb this. This is intended and
     should not be "corrected".
```

```
## 2. Network

Acceleration: alpha_theta(y,v,u) = -a v - b y - c y^3 + g u + S_alpha * N_theta(y/S_y, v/S_v, u/S_u)

- **Scales** (computed from the training record only):
  - S_y = std(y_train)
  - S_v = omega?*S_y with omega? = 2pi*70
  - S_u = std(u_train)
  - S_alpha = omega?^2*S_y
- **MLP N_theta:** 3 -> 32 -> 32 -> 1, tanh activation, flo
[... abridged ...]
 after that run.

## Revision rules, if the first results are poor

- Keep eq. (1) and the free-run test unchanged.
- Diagnose from the train free-run error and the loss history, not from the test error.
- The first levers are a longer L in stage 3, 2 substeps per sample, a lower ?, and a larger network (64 units).
- If the rollout diverges, lower the learning rate and raise ?.
```

Turn 4 — orchestrator

```
Decision 2/8 -> executor
Instruction: Implement the scientist's latest plan (Silverbox PINN) in a new Python file and run it (5-minute
     limit). Follow the numbered steps exactly: load SNLS80mV.mat, slice [42650:46746], center, 3072/1024 split,
     least-squares init of a,b,c,g, physics-prior + zero-init residual MLP (3-32-32-1 tanh, float64), RK4 with
     midpoint u, multiple-shooting stages 0-3 (optional 4) with wall-clock guards, then one free run from sample 0
      through train into test reading only u. Save the plot of measured vs predicted on the test segment to
     test_segment.png. Print the final a,b,c,g, and as the last two lines TRAIN_RMSE=<float> and TEST_RMSE=<float
     >. Return the printed output verbatim, or the full traceback if it fails.
Reason: The plan is complete, so it must now be coded and run to obtain verified RMSE values.
```

Turn 5 — executor

```
File: silverbox_pinn.py. Exit code: 0. Runtime was 207.6 s, inside the 5-minute limit. The plot was saved as
     test_segment.png.

Printed RMSE lines, exactly as printed:
TRAIN_RMSE=0.0014457934186479073
TEST_RMSE=0.0010088887947707441

Other printed values:
- Least-squares init: a=39.6, b=1.760e5, c=5.749e5, g=1.600e5.
- Final fitted values: a=41.19, b=1.848e5, c=7.4768e5, g=1.9608e5.
- Maximum |?| of the free run was 0.2068.
- The zero-predictor test RMSE was 0.0605.
- Training stage S3 hit its time budget at iteration 109 of 150.
- Stage 4 stopped after 8 iterations, at the 30 s budget.
- stderr held only a harmless PyTorch UserWarning about converting a tensor that requires grad to a float.

The free run started at sample 0 and carried its state through the train/test boundary. It read only u after the
     first sample.

[tool record]
write silverbox_pinn.py: ok
run silverbox_pinn.py: exit code 0, 210.0 s
VERIFIED printed by the script: TEST_RMSE=0.0010088887947707441, TRAIN_RMSE=0.0014457934186479073
```

Turn 6 — orchestrator

```
Decision 3/8 -> finish
Instruction: The Silverbox PINN ran and the script printed the free-run errors. The script is silverbox_pinn.py
     and the plot is test_segment.png.

Verified results, exactly as printed:
- TRAIN_RMSE=0.0014457934186479073
- TEST_RMSE=0.0010088887947707441

For scale, predicting zero scores 0.0605 V RMSE on the test segment, so the model is much better than the
     baseline.

Method:
- The model is eq. (1) as a physics prior with learnable a, b, c and g, plus a residual 3-32-32-1 tanh network
     whose last layer starts at zero.
- It was integrated with RK4 and trained by multiple shooting.
```

```
- The free run started at sample 0 with a finite-difference velocity. It carried the state through the train/test
      boundary and read only u after the first sample.

Fitted coefficients:
- Least-squares initialization: a=39.6, b=1.760e5, c=5.749e5, g=1.600e5.
- Final: a=41.19, b=1.848e5, c=7.4768e5, g=1.9608e5.

Other notes:
- The maximum |?| of the free run was 0.2068.
- The run took about 208 s.
- The last stage-3 and stage-4 runs stopped at their time budgets (iteration 109 of 150 and 8 iterations).
- stderr held only a harmless PyTorch UserWarning.
Reason: A VERIFIED executor result with both TRAIN_RMSE and TEST_RMSE exists, so the run can be finished.
```